ЗАЩИТА КАЧЕСТВА, ТОЧНОСТИ И ТЕСТИРОВАНИЯ

Единый расширенный документ по EH/Unwind Analyzer

Внутри: общая защитная позиция, тестовая архитектура, 16 CTest-сценариев, 27 признаков, 78 PE-бинарников, FP/FN/TP/TN, precision/recall/accuracy, scoring, confidence, веса findings, ложные срабатывания, пропуски и готовые ответы комиссии.

Как пользоваться документом

Этот PDF собран как главный защитный пакет по качеству. Его удобно держать открытым на защите, когда комиссия спрашивает про тестирование, точность, ложные срабатывания, обоснование score или качество продукта. Документ специально содержит несколько уровней: сначала краткая защитная модель, затем подробная методика тестирования, затем точность и scoring, затем отдельная памятка по FP/FN.

Часть	Что закрывает
Часть I. Общая защита качества	Объясняет, почему проект нельзя оценивать одним процентом, что такое quality axes, TP/FP/TN/FN, score, confidence и текущие evidence.
Часть II. Как тесты объяснить	Подробно объясняет 16 CTest-сценариев, 27 признаков, 78 бинарников, corpus, mutation fuzz и ответы комиссии.
Часть III. Качество и точность проекта	Дает методику оценки accuracy/precision/recall, false positive rate, false negative rate, dynamic event quality и scoring appendix.
Часть IV. False Positive / False Negative	Отдельно разжевывает ложные срабатывания и пропуски, связь с ML-терминами и корректные формулировки.

ЗАЩИТА КАЧЕСТВА, ТОЧНОСТИ И ТЕСТИРОВАНИЯ

Большой защитный документ по EH/Unwind Analyzer

Тестирование, точность, false positives, false negatives, scoring, веса findings, текущие доказательства качества и готовые ответы комиссии.

Главная защитная позиция
Проект является rule-based анализатором PE x64 EH/Unwind metadata, а не ML-классификатором вредоносности. Поэтому качество нельзя честно выразить одним процентом. Оно доказывается системой: CTest, корпус тестовых PE, schema/semantic validation, benign validation, mutation fuzz, dynamic validation, evidence-oriented reports и понятная методика расчета TP/FP/TN/FN на размеченном benchmark.

1. Что именно защищаем

EH/Unwind Analyzer проверяет не “вредоносность файла вообще”, а техническое состояние PE x64 EH/Unwind metadata. Он читает PE, .pdata, .xdata, RUNTIME_FUNCTION, UNWIND_INFO, handler RVA, section permissions и применяет 27 эвристических признаков. Поэтому оценка качества должна соответствовать задаче: проверяется корректность анализа metadata, а не универсальное определение malware.

Объект качества	Что значит “работает правильно”	Как доказывать
PE/parser layer	Файл читается безопасно; ошибочные поля не приводят к crash.	Unit tests, malformed samples, mutation fuzz.
.pdata/.xdata layer	RUNTIME_FUNCTION и UNWIND_INFO интерпретируются по правилам формата.	Regression samples, expected findings, diagnostic evidence.
Heuristic layer	27 признаков срабатывают при своих условиях и дают обоснованный score.	Finding-level tests, corpus coverage, scoring table.
Report layer	JSON/HTML/CSV показывают evidence и согласованные поля.	Schema validation, semantic validation, HTML checks.
Dynamic layer	Windows runtime events фиксируются при достижении соответствующих путей.	Dynamic validator, instrumented targets, dynamic corpus.

2. Почему не один процент accuracy

Один процент accuracy был бы понятным, но научно слабым. Проект решает несколько разных задач: parsing, detection, reporting, dynamic observation, robustness. У каждой задачи своя единица оценки. Accuracy без указания unit of evaluation, positive class, negative class, threshold и ground truth ничего не доказывает.

Единица оценки	Вопрос	Пример
File-level	Должен ли весь файл получить хотя бы один finding?	Файл с любым серьезным нарушением считается positive.
Record-level	Правильно ли отмечена конкретная RUNTIME_FUNCTION/UNWIND_INFO?	Одна запись имеет invalid range или parse failure.
Finding-level	Должен ли конкретный code появиться в отчете файла?	Для файла X должен появиться unwind_parse_fail.
Event-level	Должно ли runtime-событие быть зафиксировано?	Target вызывает RtlAddFunctionTable, dynamic должен это увидеть.

Готовый ответ
Если спросят “какая точность?”, отвечать: “Нужно уточнить, точность какого слоя. Глобальную accuracy без большого ground truth benchmark я не заявляю. Для текущей версии показываю инженерную валидацию, а строгую detection accuracy надо считать per finding class через TP/FP/TN/FN.”

3. Термины TP, FP, TN, FN

Эти термины часто используются в машинном обучении, но применимы к любому detector-у. Главное условие - наличие ground truth. В проекте они используются не внутри алгоритма, а для оценки качества на размеченном corpus.

Ground truth / результат	Analyzer дал сигнал	Analyzer не дал сигнал
Нарушение есть	TP: true positive. Нарушение найдено правильно.	FN: false negative. Нарушение пропущено.
Нарушения нет	FP: false positive. Ложное срабатывание.	TN: true negative. Правильное молчание.

Пример TP: в PE специально создан CountOfCodes/UWOP slots mismatch, и отчет содержит unwind_parse_fail с expected_slots/available_slots. Пример FP: легитимный JIT/runtime layout получил suspicious placement finding, хотя это нормальное поведение runtime. Пример FN: metadata и код изменены согласованно, текущие инварианты не нарушены, и finding не появляется.

4. Ground truth и почему без него нельзя говорить о FP/FN

Ground truth - эталонная разметка. Она должна отвечать: какое нарушение есть, где оно расположено, почему это нарушение, как оно подтверждено и кто проверил. Без ground truth можно сказать “есть suspicious finding”, но нельзя строго сказать “это FP” или “это FN”.

Источник ground truth	Для чего подходит	Ограничение
Synthetic PE	Точные positive samples для конкретных findings.	Не покрывает все реальные toolchains.
Manual expert review	Сложные реальные cases.	Дорого и зависит от квалификации эксперта.

Источник ground truth	Для чего подходит	Ограничение
Differential parser comparison	Проверка parser discrepancies.	Другой parser тоже может ошибаться.
Compiler/linker provenance	Benign negatives и expected normal layout.	Не всегда известно для внешних PE.
Instrumented target	Dynamic event ground truth.	Покрывает только заданный runtime path.

5. Метрики точности

Метрика	Формула	Что означает
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Доля правильных решений. Опасна при дисбалансе классов.
Precision	TP/(TP+FP)	Если analyzer сработал, насколько часто сигнал подтверждается.
Recall	TP/(TP+FN)	Если нарушение есть, насколько часто analyzer его находит.
Specificity	TN/(TN+FP)	Если нарушения нет, насколько часто analyzer не дает ложную тревогу.
FPR	FP/(FP+TN)	False positive rate: доля ложных тревог среди нормальных cases.
FNR	FN/(FN+TP)	False negative rate: доля пропусков среди positive cases.
F1	2*precision*recall/(precision+recall)	Баланс precision/recall, но не учитывает TN.

Accuracy может обманывать. Если corpus почти весь benign, инструмент с нулевым recall может иметь высокий accuracy. Поэтому для defense правильнее говорить: “качество надо считать отдельно по каждому finding class, а не одним средним числом”.

6. False positives: глубокое обоснование

False positive - это не просто “ошибка программы”. Для эвристического reverse-engineering инструмента FP часто означает, что технический pattern похож на подозрительный, но имеет benign explanation. Поэтому проект не должен автоматически писать “malware”. Он должен писать finding, evidence, severity, confidence и manual validation steps.

Источник FP	Почему возникает	Как проект снижает риск
Редкий compiler layout	Секция или handler расположены непривычно, но допустимо.	unusual placement имеет умеренный вес; требуется manual review.
Language runtime handler	Один handler обслуживает много функций легитимно.	handler_dominance имеет medium weight и context checks.
JIT/runtime metadata	RtlAddFunctionTable может быть нормальным для JIT.	Dynamic фиксирует событие, но не объявляет вредоносность.
Protector/packer benign use	Защитный runtime меняет layout без malware intent.	Отчет показывает technical evidence, а не moral verdict.
Small sample ratios	На маленькой .pdata один случай дает высокий процент.	confidence penalty small_sample_size.
Placement-only signal	Одна странная секция без других проблем.	confidence penalty placement_only_signal.

Защита от FP в проекте строится не только на коде, но и на стиле вывода: analyzer показывает, что именно обнаружено, где RVA, какая секция, какие права, какой affected_count, какие expected/observed values. Реверсер может проверить evidence, а не принимать вывод вслепую.

7. False negatives: глубокое обоснование

False negative - это пропуск реального нарушения. Для этого проекта FN возможны там, где вмешательство не нарушает текущие инварианты, где semantic mismatch требует дизассемблирования .text, или где runtime path не был достигнут во время dynamic run.

Источник FN	Пример	Как снижать
Согласованная подмена metadata	Все поля формально валидны, но смысл изменен.	Добавить semantic/prolog matching и differential analysis.
Нет disassembly .text	UWOP не соответствует реальному prolog, но структура .xdata валидна.	Добавить дизассемблерный слой.
Неисполненный путь	Handler есть, но тестовый запуск не вызвал exception.	Targeted dynamic inputs, scenario coverage.
Ненаблюдаемый API/route	Dynamic registration идет вне текущих breakpoints.	Расширить instrumentation.
Unknown toolchain	Новый формат или нетипичная layout policy.	Пополнять corpus и правила.

Сильный ответ
Наличие потенциальных FN не уничтожает проект. Это нормальное свойство любого analyzer-a. Важно, что проект честно ограничивает область: он обнаруживает 27 классов структурных/контекстных признаков и предлагает понятный план снижения FN: disassembly, per-function correlation, dynamic scenarios, расширение corpus.

8. Score и почему это не процент вредоносности

Score - инженерная шкала подозрительности 0..100. Она формируется суммой весов findings. Suspicion = score/100, но это нормировка для отчета, а не вероятность атаки. Score 25 означает “накоплено 25 баллов evidence”, а не “файл на 25% вредоносный”.

Порог	Verdict	Как объяснять
0-14	no strong static anomalies	Сильных статических признаков не найдено.
15-39	low_suspicion	Есть технические признаки, но evidence ограничено.
40-69	moderate_suspicion	Есть заметная совокупность признаков или жесткий класс проблем.
70-100	high_suspicion	Сильная/многосемейная картина, требующая приоритетной ручной проверки.

9. Почему findings имеют такие веса

Веса выбраны инженерно по силе evidence. Нарушения, которые ломают базовое чтение PE/.pdata/.xdata, получают высокие веса. Контекстные признаки placement/reuse получают средние веса, потому что могут иметь benign explanations. Combo-признаки получают дополнительные веса, потому что усиливают уже найденную картину, но не должны подменять базовое evidence.

Finding	Вес	Обоснование
pe_section_raw_oob	25	raw section выходит за файл; сильная структурная ошибка
pe_section_virtual_oob	20	virtual section выходит за SizeOfImage
pe_section_raw_overlap	15	raw overlaps, средняя неоднозначность
pe_section_virtual_overlap	20	RVA overlaps, сильная неоднозначность
pe_section_misaligned	8	alignment issue, может быть менее опасным
pe_exception_directory_oob	25	сломанный вход в .pdata
no_runtime_functions	10	может быть benign leaf-only image
invalid_ranges	10/20	End<=Begin; >1% усиливает до 20
rf_oob	10/25	function range OOB; >1% усиливает до 25
unwind_rva_oob	10/25	UnwindInfoAddress OOB; >1% усиливает до 25
rf_unsorted	15	нарушение порядка .pdata
rf_overlaps	25	один ControlPc может иметь конфликтующую запись
duplicate_begin	10	повтор begin требует проверки
unwind_parse_fail	10/25	UNWIND_INFO не читается; >1% усиливает
unwind_anomalies	10/20	аномалии .xdata; >5% усиливает
unwind_reuse_strong	20	>30% reuse при n>200
unwind_reuse_moderate	10	>10% reuse + другой static signal
handler_dominance	15	>80% одного handler при >50 handlers
unwind_in_exec_section	20	metadata в executable секции
unwind_unusual_section	8/15	нетипичная секция; >5% усиливает
unwind_writable_section	10/18	writable metadata; >5% усиливает
handler_nonexec_section	15	handler не в executable code
handler_writable_section	10	handler target writable
placement_exec_nonexec_handler_combo	10	exec metadata + nonexec handler
placement_integrity_combo	8	placement + integrity problems
reuse_dominance_combo	8	reuse + handler dominance
writable_metadata_handler_combo	10	writable metadata + writable handler

10. Confidence и weakening rules

Confidence - это не accuracy. Это внутренняя оценка надежности вывода. Она растет, когда evidence приходит из разных семейств и есть сильные findings, и снижается, если evidence одиночное, sample маленький или сигнал только placement-only. Это защищает от чрезмерной уверенности и снижает риск false positives.

Правило	Эффект	Зачем
per evidence family	+0.11	разные семьи evidence сильнее одной семьи
per strong finding	+0.08	high findings усиливают вывод
per medium finding	+0.04	medium findings добавляют умеренную уверенность
hard table bonus	+0.08	жесткие .pdata проблемы надежнее контекстных
hard unwind bonus	+0.08	parse/anomaly .xdata важны
single signal penalty	-0.12	один слабый сигнал не должен переубеждать
small sample penalty	-0.06	маленькая .pdata искажает ratios
placement-only penalty	-0.10	placement может быть benign
single-family penalty	-0.08	нет cross-family corroboration

11. Тестовая архитектура проекта

Тестирование построено как quality gate. 16 CTest-сценариев - это не 16 бинарников и не 16 признаков, а 16 автоматизированных проверочных сценариев. Один сценарий может запускать несколько проверок, несколько samples или 120 mutations.

CTest-сценарий	Что проверяет
unit_unwind	низкоуровневый разбор UNWIND_INFO, UWOP slots, operands, chain
unit_heuristics	базовые эвристики и счетчики findings
unit_heuristics_mitigations	снижение confidence при слабом evidence
unit_heuristics_context	section permissions, handler/reuse context
unit_heuristics_combos	комбинированные признаки
unit_core_edges	границы, OOB, безопасная логика ядра
unit_report_json	JSON contract
unit_report_html	HTML report и matrix 27 features
unit_matrix_report	матрица признаков
regression_samples	end-to-end запуск samples + schema/semantic validation
dynamic_validator	валидность dynamic evidence reports
report_semantics	смысловые инварианты JSON
release_metadata	версии и metadata релиза
release_hygiene	чистота поставки
cli_contract	help/version/exit codes/atomic writes/redaction
mutation_fuzz	120 мутаций PE, crash-free controlled behavior

12. 16 тестов, 27 признаков, 78 бинарников

Число	Что это	Как объяснять
16	CTest-сценарии	Автоматические тестовые сценарии качества кода и контрактов.
27	Findings/features	Диагностическая модель анализа.
78	Тестовые PE	Расширенный corpus: 60 static, 9 dynamic, 9 mixed.
120	Mutation fuzz cases	Проверка robustness.
106	Benign real corpus из release docs	Проверка отсутствия наблюдаемых non-zero scores на данном benign наборе.

Эти числа не должны совпадать. Один PE может проверять несколько findings, один CTest может запускать много samples, один finding может проверяться unit-тестом, corpus sample и отчетом одновременно.

13. Текущие доказательства качества

Evidence	Что зафиксировано	Как защищать
Локальный CTest	16/16 passed	Кодовый quality gate проходит.
Extended corpus	78 PE processed, reports generated	Есть воспроизводимая демонстрация scenarios.
HTML matrix	27 features visible active/inactive	Комиссия видит не только сработавшие признаки, но и всю модель.
Key diagnostics	expected/available slots/bytes/file offset	Отчет показывает evidence, а не магический verdict.
Release docs	14/14 static labeled valid; 106/106 benign valid; non-zero scores 0; 174795 runtime functions parsed	Сильный validation signal, но не глобальная accuracy.
Mutation fuzz	120 deterministic mutations	Устойчивость к поврежденным PE.

14. Как строить строгий benchmark дальше

- Определить unit of evaluation: file/record/finding/event.
- Для каждого из 27 finding codes собрать positive и negative samples.
- Сделать ground truth: synthetic construction, manual review, differential parsers, compiler provenance.
- Запускать analyzer одинаковой командой, сохранять JSON и версию schema.
- Считать TP/FP/TN/FN отдельно по каждому finding code.
- Считать precision, recall, specificity, FPR, FNR, F1.
- Отдельно считать benign false-positive rate на real corpus.
- Отдельно считать dynamic event recall на instrumented Windows targets.
- Каждый FP/FN превращать в улучшение rule, threshold, mitigation или test sample.

15. Вопросы комиссии и ответы

Какая точность проекта?

Глобальную accuracy без большого размеченного benchmark я не заявляю. Текущая оценка - инженерная: CTest, corpus, reports, benign validation, fuzz. Строгая точность считается per finding class через TP/FP/TN/FN.

Почему не 100%, если тесты прошли?

Тесты доказывают корректность на заданных сценариях. Они не доказывают абсолютное отсутствие FP/FN на всех PE мира.

Что такое false positive в вашем проекте?

Finding появился, но ground truth говорит, что нарушения нет. Например, редкий benign runtime layout похож на suspicious placement.

Как снижаете false positives?

Context rules, severity, confidence penalties, benign corpus validation, manual validation steps, evidence-oriented reports.

Что такое false negative?

Нарушение есть, но текущие правила его не обнаружили. Например, согласованная metadata или mismatch, требующий disassembly .text.

Как снижаете false negatives?

Расширение правил, disassembly/prolog matching, targeted dynamic, новый corpus, per-finding benchmark.

Почему score не процент вредоносности?

Score - сумма инженерных весов evidence 0..100. Он нормируется в suspicion, но не является ML probability и не заменяет экспертный verdict.

Почему такие веса findings?

Веса отражают силу evidence: hard structural/unwind violations выше, placement/reuse средние из-за benign alternatives, combo усиливают связки.

Почему нет ML?

Форматные инварианты PE/EH/Unwind лучше проверяются объяснимыми правилами. ML можно добавить позже для triage, но не вместо строгого parser-a.

16. Готовый устный ответ

“Качество проекта я оцениваю как систему, а не одним процентом. Analyzer не является нейросетевой моделью вредоносности; это rule-based инструмент анализа PE x64 EH/Unwind metadata. Поэтому качество делится на корректность parser-a, корректность 27 findings, устойчивость к malformed input, качество JSON/HTML отчетов, dynamic validation и релизную готовность. Текущий quality gate прошел 16/16 CTest, расширенный корпус содержит 78 PE, отчеты сформированы во всех форматах, а release-документы фиксируют labeled/benign validation.”

“Если строго говорить о точности, нужно строить benchmark: определить ground truth, positive/negative classes и unit of evaluation, затем считать TP, FP, TN, FN, precision, recall, specificity и false-positive rate отдельно по каждому finding code. Я сознательно не заявляю необоснованную 100% accuracy, потому что это было бы некорректно. Вместо этого показываю воспроизводимое тестирование, evidence в отчетах и понятный план строгой статистической оценки.”

“Ложные срабатывания возможны, потому что некоторые benign runtime/toolchain layouts похожи на suspicious patterns. Проект снижает FP через контекст, confidence penalties и manual validation. Ложные пропуски тоже возможны, если вмешательство не нарушает текущие инварианты; для снижения FN нужны disassembly/prolog matching и более глубокая static/dynamic correlation.”

17. Итоговая защита

Защитная позиция зрелая: проект не переобещает. Он не говорит “я определяю любое вредоносное вмешательство с точностью 100%”. Он говорит: “я воспроизводимо анализирую EH/Unwind metadata, нахожу 27 классов структурных и контекстных аномалий, показываю evidence, тестирую parser/reports/CLI/robustness/dynamic validation и имею понятную методику дальнейшего измерения precision/recall”. Именно это делает проект защищаемым перед жесткой комиссией.

КАК ТЕСТЫ ОБЪЯСНИТЬ

Методическое объяснение тестирования EH/Unwind Analyzer для комиссии

Документ подготовлен для защиты проекта: как объяснять качество, покрытие тестами, связь 16 CTest-сценариев, 27 эвристических признаков и 78 тестовых PE-бинарников.

Короткая формула для защиты
В проекте реализовано 27 диагностических признаков. Они проверяются многоуровневой системой из 16 автоматизированных CTest-сценариев и расширенным корпусом из 78 PE-бинарников. Числа не обязаны совпадать, потому что признак - это диагностическое правило, CTest-тест - это сценарий проверки кода, а бинарник - это тестовый входной артефакт.

Актуальная проверка: 13 августа 2026. CTest: 16/16 passed. Расширенный корпус: 78 PE-образцов обработаны, сгенерированы HTML/JSON/CSV/Ghidra-CSV отчеты.

1. Зачем нужен этот документ

На защите вопрос о тестировании обычно является вопросом не только о количестве тестов. Комиссия хочет понять, насколько проект контролируем, воспроизводим и устойчив. Поэтому важно объяснять тестирование не как список команд, а как систему доказательств качества. Для данного проекта качество подтверждается на нескольких уровнях: низкоуровневые unit-тесты, регрессионные PE-образцы, проверка отчетов, проверка CLI-контракта, fuzz/mutation-проверка и контроль релизной поставки.

Главная ошибка в устном объяснении - сказать: “у меня 27 признаков, но 16 тестов”. Такая фраза звучит как несоответствие. Правильнее сказать: “у меня 27 диагностических признаков, и они проверяются 16 тестовыми сценариями разного уровня; один сценарий может покрывать несколько признаков, а расширенный корпус из 78 бинарников демонстрирует работу признаков на конкретных PE-входах”.

> Как сказать комиссии
> Тестирование построено не по принципу “один признак - один тест”, а по принципу test design: отдельно проверяется парсер, отдельно эвристический слой, отдельно формат отчетов, отдельно CLI-контракт, отдельно устойчивость к поврежденным PE и отдельно демонстрационный корпус. Это более правильная структура для системного инструмента анализа бинарных файлов.

2. Три разных числа: 16, 27 и 78

Число	Что это	Как объяснять
16	CTest-сценарии	Автоматические проверки, зарегистрированные в CMake/CTest. Это тестовые программы и Python-сценарии, которые запускаются как quality gate.
27	Эвристические признаки	Диагностические правила модели анализа. Они описывают, какие аномалии инструмент умеет обнаруживать в PE, .pdata, .xdata, handler placement и комбинациях признаков.
78	Тестовые PE-бинарники	Входные образцы, специально подготовленные для демонстрации нормальных, статических, динамических и смешанных сценариев.

Эти числа относятся к разным слоям. Поэтому требование “должно быть 27 тестов, потому что 27 признаков” некорректно. Один CTest-сценарий может проверять сразу несколько признаков, а один бинарник может активировать несколько findings одновременно. В обратную сторону тоже возможно: один признак может проверяться и unit-тестом, и корпусным бинарником, и проверкой отчета.

Наиболее сильное объяснение звучит так: CTest отвечает за автоматический контроль кода; 27 признаков отвечают за функциональную полноту диагностической модели; 78 бинарников отвечают за демонстрационное и регрессионное покрытие входных PE-сценариев.

3. Что именно считается качеством для этого проекта

Для EH/Unwind Analyzer качество нельзя свести только к “программа запускается”. Инструмент анализирует бинарный формат, где ошибки чтения на несколько байтов могут полностью изменить смысл результата. Поэтому критерии качества делятся на несколько групп.

- Корректность parsing layer. Парсер должен правильно читать PE headers, sections, Exception Directory, RUNTIME_FUNCTION и UNWIND_INFO, включая CountOfCodes, дополнительные UWOP slots, padding, flags, handler tail и CHAININFO.

- Безопасность на поврежденных входах. При malformed PE инструмент не должен падать, зависать или читать за пределами буфера. Он должен либо создать частичный отчет, либо корректно вернуть код ошибки.
- Корректность эвристического слоя. Finding должен появляться только тогда, когда выполнено условие признака. Счетчики, severity и итоговая оценка должны быть согласованы между собой.
- Объяснимость отчета. JSON и HTML должны показывать не только итог, но и доказательства: RVA, секцию, file offset, affected count, причину parse failure, значения expected/available и матрицу всех 27 признаков.
- Стабильность интерфейса. CLI должен иметь предсказуемые exit codes, help/version, валидацию аргументов, атомарную запись отчетов и защиту от утечки target arguments.
- Релизная готовность. В поставке не должно быть временного мусора, несовместимых версий schema, случайных артефактов сборки и файлов, которые мешают заказчику использовать инструмент.

4. Тестовая архитектура

Тестирование построено слоями. Нижний слой проверяет атомарные функции ядра. Средний слой проверяет связку “анализатор -> отчет -> схема -> семантические инварианты”. Верхний слой проверяет пользовательский контракт CLI и релизную поставку. Отдельно стоит robustness testing: mutation_fuzz имитирует поврежденные PE и проверяет, что инструмент остается управляемым.

Уровень	Что проверяется	Почему это важно
Unit	Парсер, эвристики, матрица, JSON/HTML генерация.	Локализует ошибки в конкретном модуле и быстро показывает, что сломалось.
Integration	Запуск реального analyzer на PE-образцах, проверка JSON schema и semantic validator.	Проверяет, что модули работают вместе, а не только по отдельности.
Corpus regression	Подготовленные PE-бинарники с известными нарушениями.	Доказывает, что признаки обнаруживаются на входных файлах, близких к реальному формату.
CLI contract	Публичные флаги, ошибки аргументов, exit codes, atomic writes.	Защищает пользователя и будущий backend от непредсказуемого поведения.
Robustness / fuzz	120 мутаций входных PE.	Проверяет отказоустойчивость к поврежденным файлам.
Release hygiene	Метаданные, чистота поставки, версия schema.	Показывает готовность к передаче заказчику.

5. Полный список 16 CTest-сценариев

Ниже важно объяснять не “почему их всего 16”, а что каждый из них является тестовым сценарием. Сценарий может содержать много assert-ов, запусков инструмента, проверок файлов и проверок инвариантов.

CTest	Уровень	Что проверяет	Зачем нужен
unit_unwind	Unit	Проверяет низкоуровневый разбор x64 UNWIND_INFO: заголовок, CountOfCodes, операции UWOP, дополнительные slots, tail и chain-логику.	Доказывает, что базовый парсер не строит выводы из неправильно прочитанных байтов.
unit_heuristics	Unit	Проверяет расчет базовых статических эвристик и счетчиков finding-ов.	Нужен, чтобы 27 признаков работали как модель анализа, а не как текстовые метки.

CTest	Уровень	Что проверяет	Зачем нужен
unit_heuristics_mitigations	Unit	Проверяет контекстные смягчения и предотвращение завышенной тревожности.	Защищает от ситуации, когда любой нестандартный файл автоматически считается подозрительным.
unit_heuristics_context	Unit	Проверяет учет секций, прав доступа, handler targets, reuse и дополнительного контекста.	Показывает, что анализатор смотрит на взаимосвязи, а не только на одно поле.
unit_heuristics_combos	Unit	Проверяет комбинированные признаки: когда несколько слабых сигналов вместе дают более сильный вывод.	Это важно для защиты, потому что проект анализирует корреляции между признаками.
unit_core_edges	Unit	Проверяет граничные случаи ядра: адресные преобразования, диапазоны, ошибки маппинга и защиту от выхода за границы.	Без этого malformed PE мог бы ломать анализ или давать ложные выводы.
unit_report_json	Unit / contract	Проверяет JSON-отчет: обязательные поля, числовые счетчики, структура evidence и machine-readable контракт.	JSON - основной интерфейс для будущей БД, backend и frontend.
unit_report_html	Unit / presentation	Проверяет HTML-отчет, включая наличие матрицы 27 признаков и диагностических разделов.	Комиссии можно показать, что отчет тестируется как часть продукта, а не делается “на глаз”.
unit_matrix_report	Unit / presentation	Проверяет итоговую матрицу, статусы признаков и читаемость сводки.	Матрица помогает объяснить, что сработало, что не сработало, и почему.
regression_samples	Integration	Прогоняет минимальный набор PE-образцов, сверяет JSON schema, semantic validator и ожидаемые поля отчета.	Это быстрый контроль: после изменения кода старые гарантии не должны исчезнуть.
dynamic_validator	Integration / dynamic	Проверяет валидатор динамических отчетов для VEH, UEF, VCH и RTL function table сигналов.	Динамический слой проверяется по структуре evidence и корректности обязательных runtime-полей.
report_semantics	Integration / report	Проверяет не только схему JSON, но и смысловые инварианты: размеры функций, счетчики, секционные права, dynamic-сводку.	Схема ловит форму, semantic validator ловит противоречия внутри отчета.
release_metadata	Release	Проверяет метаданные релиза, версии, schema_version и служебную информацию.	Защищает передачу заказчику от несогласованных версий.
release_hygiene	Release	Проверяет чистоту релизной поставки: отсутствие лишних временных файлов, мусора и небезопасных артефактов.	Это тест не алгоритма, а готовности продукта к передаче.
cli_contract	CLI / integration	Проверяет help/version, ошибки аргументов, exit codes, atomic output, redaction target arguments, partial reports.	Показывает, что инструмент предсказуемо ведет себя как CLI-продукт.
mutation_fuzz	Robustness	Запускает 120 детерминированных мутаций PE-файлов: битовые инверсии, обрезки, вставки, удаление блоков, порчу e_lfanew.	Доказывает устойчивость парсера к поврежденным входам: он должен отказать корректно, а не падать.

6. Почему 16 тестов достаточно осмысленно для 27 признаков

Число CTest-тестов не равно числу проверок внутри них. Например, mutation_fuzz в CTest считается одним тестом, но внутри запускает 120 мутационных случаев. regression_samples считается одним тестом, но внутри запускает несколько PE-образцов, строит JSON-отчеты, проверяет schema, semantic validator и ожидаемые значения. unit_report_html считается одним тестом, но проверяет наличие матрицы всех 27 признаков в HTML.

Поэтому показатель качества здесь - не совпадение чисел, а наличие traceability: для каждого диагностического признака есть место, где он проверяется логикой, и есть бинарный сценарий, который демонстрирует его срабатывание.

> Готовый ответ
> Если спросят: “почему признаков 27, а тестов 16?”, отвечать так: “Потому что 27 - это количество диагностических признаков модели, а 16 - количество автоматизированных тестовых сценариев. Один сценарий покрывает несколько признаков, а отдельные сценарии проверяют не признаки, а свойства продукта: schema, CLI, устойчивость к malformed input, атомарность записи отчетов и релизную чистоту. Дополнительно есть 78 тестовых бинарников как входной корпус для демонстрации и регрессии.”

7. Расширенный корпус из 78 PE-бинарников

Расширенный корпус нужен не вместо CTest, а вместе с CTest. CTest - это автоматический quality gate разработчика. Корпус бинарников - это тестовые данные и демонстрационная база для защиты, отчета и проверки конкретных аномалий. В текущей версии корпуса: 60 static, 9 dynamic и 9 mixed образцов.

Категория	Количество	Назначение
static	60	Проверка PE layout, Exception Directory, .pdata, .xdata, handler placement, reuse/dominance и combo-признаков без запуска target.
dynamic	9	Сценарии для runtime-сигналов: VEH, UEF, VCH, RTL function table и исключения при исполнении.
mixed	9	Сценарии, где статическая странность должна сопоставляться с динамическим поведением.
Итого	78	Расширенный демонстрационный и регрессионный PE-корпус.

По корпусу были сгенерированы 78 HTML, 78 JSON, 78 CSV и 78 Ghidra-CSV отчетов. Это важно: проверяется не только факт запуска анализатора, но и способность выпускать все форматы вывода для каждого образца.

Четыре специальных образца с некорректной Exception Directory ожидаемо возвращают код частичного анализа 1. Это не провал тестирования. Это проверка правильного поведения на malformed input: инструмент не падает, а формирует частичный структурный отчет и фиксирует соответствующий finding.

8. Матрица трассировки 27 признаков

Traceability matrix показывает связь между диагностическим признаком, смыслом проверки и тестовым покрытием. Именно эту таблицу удобно использовать на защите, если комиссия спрашивает: “а где доказательство, что все признаки проверялись?”.

Код признака	Слой	Что означает	Чем покрывается
pe_section_raw_oob	PE layout	Raw-диапазон секции выходит за границы файла.	unit_core_edges, cli_contract, corpus 21-22
pe_section_virtual_oob	PE layout	Virtual-диапазон секции выходит за SizeOfImage.	unit_core_edges, corpus 23-24
pe_section_raw_overlap	PE layout	Физические диапазоны секций пересекаются.	unit_core_edges, corpus 25
pe_section_virtual_overlap	PE layout	RVA-диапазоны секций пересекаются.	unit_core_edges, corpus 26

Код признака	Слой	Что означает	Чем покрывается
pe_section_misaligned	PE layout	Нарушены alignment-поля или выравнивание секций.	cli_contract, corpus 27-28
pe_exception_directory_oob	PE layout	Exception Directory не мапится или имеет некорректный размер.	cli_contract, corpus 29-32
no_runtime_functions	.pdata	В PE x64 нет читаемых RUNTIME_FUNCTION записей.	heuristics, corpus 33
invalid_ranges	.pdata	BeginAddress >= EndAddress.	unit_heuristics, corpus 04, 34-35
rf_oob	.pdata	Диапазон функции выходит за границы image.	unit_heuristics, corpus 05, 36-37
unwind_rva_oob	.pdata -> .xdata	UnwindInfoAddress не указывает на допустимые байты.	regression_samples, corpus 03, 19-20, 38-39
rf_unsorted	.pdata	Таблица RUNTIME_FUNCTION не отсортирована по BeginAddress.	unit_heuristics, corpus 06, 40, 58
rf_overlaps	.pdata	Диапазоны функций пересекаются.	unit_heuristics, corpus 06, 41, 74
duplicate_begin	.pdata	Несколько записей имеют одинаковый BeginAddress.	unit_heuristics, corpus 06, 42
unwind_parse_fail	.xdata	UNWIND_INFO не может быть надежно разобран.	regression_samples, corpus 03, 38-44, 77
unwind_anomalies	.xdata	Формально прочитанная metadata содержит внутренние аномалии.	unit_unwind, corpus 07-08, 45-50, 78
unwind_reuse_strong	Reuse	Сильное повторное использование одного UnwindInfoRVA.	unit_heuristics_combos, corpus 14, 59
unwind_reuse_moderate	Reuse	Умеренное повторное использование UnwindInfoRVA.	unit_heuristics_context, corpus 58
handler_dominance	Handler	Один handler доминирует над большим числом функций.	unit_heuristics_context, corpus 14, 60-61
unwind_in_exec_section	Placement	UNWIND_INFO размещен в исполняемой секции.	unit_heuristics_context, corpus 10, 54, 70
unwind_unusual_section	Placement	UNWIND_INFO находится в нетипичной секции.	unit_heuristics_context, corpus 09, 71
unwind_writable_section	Placement	UNWIND_INFO находится в writable-секции.	regression_samples, corpus 11-12, 69
handler_nonexec_section	Handler placement	Handler RVA ведет в неисполняемую область.	unit_heuristics_combos, corpus 10, 51-52, 60, 72
handler_writable_section	Handler placement	Handler target находится в writable-секции.	unit_heuristics_context, corpus 12, 53
placement_exec_nonexec_handler_combo	Combo	Комбинация executable metadata и non-executable handler.	unit_heuristics_combos, corpus 10
placement_integrity_combo	Combo	Связка проблем размещения и целостности metadata.	unit_heuristics_combos, corpus 13
reuse_dominance_combo	Combo	Связка reuse и handler dominance.	unit_heuristics_combos, corpus 14
writable_metadata_handler_combo	Combo	Связка writable metadata и writable handler.	unit_heuristics_combos, corpus 12

9. Как работает regression_samples

Тест regression_samples - это один CTest-тест, но внутри он выполняет полноценный end-to-end контроль. Он берет компактный набор seed PE из tests/samples, при необходимости пересобирает

отсутствующие samples, запускает analyzer в режимах --dump и --heuristics --json, затем проверяет JSON report.schema.json, semantic validator и ожидаемые значения из manifest.

Смысл этого теста - не заменить все 78 демонстрационных бинарников, а дать быстрый регрессионный gate. Он нужен при каждой правке кода: если разработчик случайно изменил scoring, сломал JSON, нарушил semantic invariant или перестал находить базовую аномалию, regression_samples должен это обнаружить.

> Как объяснить разницу между 6 regression samples и 78 бинарниками
> Внутренний CTest regression_samples использует небольшой быстрый набор для постоянного контроля сборки. Расширенный набор из 78 PE - это более широкий демонстрационный и регрессионный корпус, по которому дополнительно генерируются отчеты. Это нормальная практика: быстрый обязательный gate плюс расширенный корпус для покрытия сценариев и презентации.

10. Проверка отчетов: почему это отдельная часть качества

Для данного проекта отчет - не вторичный красивый файл, а один из основных результатов анализа. Если analyzer правильно нашел проблему, но отчет не показывает поле, RVA, секцию или причину, ценность для reverse engineer-а падает. Поэтому тестирование отчетов вынесено в отдельные сценарии: unit_report_json, unit_report_html, unit_matrix_report, report_semantics.

JSON проверяется как машинный контракт: будущий backend, база данных или frontend должны получать стабильную структуру. HTML проверяется как экспертный отчет: человек должен видеть summary, ключевые диагностические ошибки, coverage статических проверок, матрицу 27 признаков и таблицу RUNTIME_FUNCTION. Semantic validator проверяет, что отчет не только формально валиден, но и внутренне непротиворечив.

Проверка	Что ловит
JSON schema	Отсутствие обязательных полей, неправильные типы, нарушение версии schema.
Semantic validator	Несогласованные счетчики, неправильный size функции, противоречия между section flags и derived fields.
HTML unit test	Отсутствие ключевых разделов, ошибки escaping, отсутствие матрицы 27 признаков.
Matrix report test	Неправильное отображение активных/неактивных признаков и итоговой матрицы.

11. CLI contract как тест продукта

Комиссия может спросить: “это тесты алгоритма или продукта?”. Важно ответить: есть и то, и другое. cli_contract проверяет именно продуктовый слой: как инструмент ведет себя для пользователя и для будущей интеграции с backend.

- Проверяются --help и --version, включая строку Report schema: 1.2.0.
- Проверяются invalid arguments: неизвестная опция, неправильный --limit, --max-chain-depth, --timeout-ms, --trace-steps.
- Проверяются exit codes: 0 для успешного анализа, 1 для частичного структурного отчета на malformed input, 2 для ошибки использования CLI.
- Проверяется atomic output: после записи отчетов не должны оставаться временные .tmp-файлы.

- Проверяется redaction target arguments: потенциально чувствительные аргументы target-процесса не должны попадать в metadata отчета.
- Проверяется создание JSON, обычного CSV и Ghidra CSV.

12. Mutation fuzz: зачем он нужен

Mutation fuzz - это проверка устойчивости, а не проверка “нашел конкретный признак”. Скрипт берет seed PE-образцы и делает 120 детерминированных повреждений: инвертирует случайные биты, затирает фрагменты, обрезает файл, вставляет случайные байты, удаляет диапазоны, портит e_lfanew и ключевые позиции файла.

Ожидаемое поведение: analyzer может принять мутированный файл и создать JSON, либо корректно отвергнуть его кодом 1. Недопустимое поведение: crash, timeout, invalid JSON после успешного кода, нарушение schema_version или чтение за пределами буфера.

> Как сказать комиссии
> Fuzz-тестирование показывает, что парсер не рассчитан только на “идеальные учебные файлы”. Он устойчив к поврежденным входам и ведет себя контролируемо: либо строит отчет, либо корректно отказывает без падения процесса.

13. Динамический слой и его тестирование

Динамическая часть проекта относится к Windows runtime: debug events, VEH, VCH, UEF, exceptions, handler hits, RtlAddFunctionTable и RtlInstallFunctionTableCallback. На macOS или Linux полноценный dynamic run Windows-процесса невозможен без Windows-среды, поэтому динамический слой разделяется на две части: проверка структуры/валидатора dynamic reports и практический прогон на Windows.

Тест dynamic_validator проверяет, что валидатор отличает корректный dynamic evidence от некорректного: VEH registration/hit, UEF set/hit, VCH registration/hit, RTL function table calls. Отдельно проверяется negative case: если report timed_out, валидатор не должен принимать его как нормальный успешный динамический отчет.

Если комиссия спрашивает, почему dynamic на macOS не подтверждает runtime, честный ответ: dynamic layer технически привязан к Windows debugging/runtime API. На macOS можно проверить static, формат отчетов и валидатор dynamic evidence, но фактические handler hits и runtime exception routes должны проверяться на Windows. Это не дефект архитектуры, а природа предметной области.

14. Что означают exit codes при тестировании

Код	Смысл	Как объяснять
0	Анализ выполнен успешно.	Файл прочитан, отчет создан, критической ошибки использования CLI нет.
1	Частичный анализ или контролируемая ошибка анализа входного файла.	Например, Exception Directory malformed. Это ожидаемо для специально сломанных PE: инструмент должен создать частичный отчет и finding.
2	Ошибка использования CLI.	Нет входного файла, неизвестная опция, недопустимое значение параметра.

На защите важно не говорить “четыре бинарника упали”. Они не упали. Они вернули ожидаемый код 1, потому что были специально повреждены на уровне Exception Directory. Разница

принципиальная: падение - это crash или неконтролируемое завершение, а код 1 с частичным отчетом - это заданное поведение инструмента.

15. Методика тестировщика: как это описывать научно

Работу над тестированием можно описать как построение test basis, test design и evidence collection. Test basis - это требования к анализатору: корректно читать PE x64 EH/Unwind metadata, выявлять структурные аномалии, формировать отчеты и устойчиво обрабатывать malformed input. Test design - это разбиение требований на уровни проверки: unit, integration, corpus, CLI, fuzz, release. Evidence collection - это сохраненные результаты: CTest output, manifest_extended.json, сгенерированные отчеты, schema validation и semantic validation.

Этап	Что делается	Результат
1. Требование	Формулируется проверяемое свойство: например, CountOfCodes должен покрывать все UWOP operand slots.	Появляется критерий ожидаемого поведения.
2. Тестовый вход	Создается PE, где UWOP_ALLOC_LARGE требует 3 slots, а CountOfCodes равен 1.	Появляется бинарник 77_static_diag_uwop_slots_beyond_count.exe.
3. Запуск	Analyzer запускается с --heuristics и выводит JSON/HTML/CSV.	Получается машинный и человекочитаемый отчет.
4. Assert	Проверяется finding unwind_parse_fail, diagnostic field, expected_slots/available_slots, отсутствие crash.	Фиксируется корректность реакции.
5. Объяснение	В HTML поднимается блок “Ключевые диагностические ошибки”.	Комиссия видит не магический итог, а доказательную цепочку.

16. Что тестирование доказывает, а что не доказывает

Хорошая защита не должна преувеличивать. Тесты доказывают корректность реализации на заданной модели требований и на подготовленных классах входов. Они не доказывают математически, что любой PE-файл в мире будет классифицирован идеально. Для бинарного анализа это нормальная инженерная позиция.

Доказывается	Не доказывается
Парсер обрабатывает нормальные и специально поврежденные PE без неконтролируемого падения в проверенных сценариях.	Абсолютная корректность на всех возможных PE, packer-ах, obfuscator-ах и compiler output.
27 признаков присутствуют в модели отчета и имеют тестовое/корпусное покрытие.	Что этих 27 признаков достаточно для любой вредоносной техники.
JSON/HTML/CSV/Ghidra-CSV формируются и проверяются автоматикой.	Что каждый внешний пользователь интерпретирует отчет без предметных знаний.
Dynamic evidence валидируется структурно, а фактический runtime-прогон должен выполняться на Windows.	Что dynamic layer полноценно исполняется на macOS/Linux без Windows runtime.

Честный ответ про точность

Это не ML-модель и не антивирусный классификатор. Поэтому “точность модели” нельзя честно заявлять как accuracy на массовом датасете без размеченного реального корпуса. В проекте корректнее говорить о функциональной точности на подготовленном регрессионном корпусе: ожидаемые признаки для тестовых PE совпадают с полученными отчетами, а устойчивость проверяется fuzz/mutation-тестированием.

17. Как отвечать на типовые вопросы комиссии

Почему 16 тестов, если признаков 27?

Потому что тест - это сценарий проверки, а признак - это диагностическое правило. Один сценарий может проверять несколько признаков, а часть тестов проверяет не признаки, а свойства продукта: CLI, JSON schema, HTML, release hygiene, fuzz robustness.

Почему не сделать один тест на каждый finding?

Можно, но это не единственный и не всегда лучший дизайн. В системном анализаторе важнее покрыть слои: parser, heuristics, reports, CLI, robustness. При этом traceability matrix показывает, где каждый finding проверяется.

Что доказывают 78 бинарников?

Они являются controlled test corpus: каждый PE создан или выбран для демонстрации конкретного нормального, статического, динамического или смешанного сценария. По ним сгенерированы отчеты, поэтому можно показать evidence для каждого класса аномалий.

Почему regression_samples внутри CTest использует меньше образцов, чем расширенный корпус?

Потому что это быстрый gate для каждой сборки. Расширенный корпус используется как более тяжелый демонстрационный и регрессионный набор. Это обычная практика: быстрые обязательные тесты плюс расширенные проверки.

Что будет, если файл сильно поврежден?

Инструмент должен либо сформировать частичный отчет с finding, либо завершиться контролируемым кодом. mutation_fuzz проверяет именно это: отсутствие crash, timeout и invalid successful report.

Как проверяется отчет?

Через JSON schema, semantic validator и unit-тесты HTML/матрицы. Отчет считается частью продукта, потому что reverse engineer принимает решение по evidence, а не только по exit code.

Динамика проверена?

Динамический слой зависит от Windows runtime/debugging API. На macOS проверяются static, report contracts и dynamic report validator. Фактические handler hits и runtime routes должны проверяться в Windows-среде.

Почему некоторые malformed binaries возвращают code 1?

Это ожидаемый partial-analysis code. Файл специально поврежден так, что Exception Directory не может быть корректно прочитана. Инструмент не падает, а фиксирует проблему и создает частичный отчет.

Можно ли по тестам сказать, что проект готов?

Можно сказать, что проект прошел текущий автоматизированный quality gate: 16/16 CTest, расширенный корпус обработан, отчеты сгенерированы. Но промышленное утверждение о точности на рынке потребовало бы отдельный размеченный real-world corpus.

Что является главным доказательством качества?

Не одно число, а совокупность: unit-тесты парсера, тесты эвристик, regression samples, schema/semantic validation, CLI contract, fuzz на 120 мутаций, 78 PE-образцов и проверенные форматы отчетов.

18. Готовый устный текст на 1-2 минуты

“Тестирование проекта построено многоуровнево. Я не считал качество только количеством тестов, потому что в таком инструменте есть разные сущности: 27 эвристических признаков, 16 автоматизированных CTest-сценариев и 78 тестовых PE-бинарников. 27 признаков - это диагностическая модель анализа. 16 CTest-сценариев - это автоматический gate, который проверяет парсер, эвристики, JSON и HTML отчеты, CLI-контракт, semantic validator, динамический валидатор, релизную чистоту и устойчивость через mutation fuzz. 78 бинарников - это расширенный controlled corpus, где каждый файл демонстрирует отдельный нормальный или поврежденный сценарий. По ним сгенерированы отчеты во всех форматах: HTML, JSON, CSV и Ghidra CSV.”

“Поэтому числа 16, 27 и 78 не должны совпадать. Они относятся к разным уровням качества. Один CTest-сценарий может проверять несколько признаков, а один бинарник может активировать несколько findings. Чтобы показать связь, в проекте есть матрица 27 признаков и расширенный manifest, где описано, что сломано в каждом PE. Последний прогон показал 16/16 passed, а расширенный корпус из 78 PE был обработан и по нему сгенерированы отчеты.”

19. Практический чек-лист перед защитой

- Показать CTest output: 16/16 passed.
- Показать папку с 78 тестовыми бинарниками: static/dynamic/mixed.
- Открыть HTML-отчет с матрицей 27 признаков.
- Открыть диагностический пример 77_static_diag_uwop_slots_beyond_count.html и показать expected_slots=3; available_slots=1.
- Объяснить, что code 1 для malformed Exception Directory - это не crash, а ожидаемый partial-analysis behavior.
- Подчеркнуть, что JSON является контрактом для будущей БД/backend/frontend.
- Отдельно сказать, что dynamic runtime требует Windows-среду, потому что использует Windows exception/debugging semantics.

20. Итог

Тестирование проекта следует объяснять как систему контроля качества, а не как набор случайных запусков. На нижнем уровне проверяются parsing и эвристики. На среднем - отчеты, schema, semantic consistency и регрессионные PE. На верхнем - CLI, релизная поставка и robustness. Расширенный корпус из 78 бинарников нужен для демонстрации покрытия признаков и для воспроизводимого анализа. 16 CTest-сценариев являются автоматическим quality gate, а 27 признаков - функциональной моделью анализа. Именно такое разделение делает объяснение профессиональным и защищаемым.

КАЧЕСТВО И ТОЧНОСТЬ ПРОЕКТА

Полная защита по вопросам accuracy, precision, recall, FP/FN и инженерной надежности EH/Unwind Analyzer

Материал рассчитан на ситуацию, когда комиссия подробно спрашивает: как доказано качество, как оценить точность, что такое ложные срабатывания, почему нельзя назвать один процент и как построить строгую валидацию.

Главная позиция
Проект не является ML-классификатором вредоносности. Это rule-based анализатор PE x64 EH/Unwind metadata. Поэтому качество оценивается не одним числом, а системой метрик: parser correctness, finding-level precision/recall, benign false-positive rate, dynamic event recall, report validity, robustness, performance, portability and release readiness.

1. Почему нельзя отвечать одним процентом

Вопрос “какая точность?” выглядит простым, но для такого проекта он неполный. Точность зависит от того, какую именно задачу мы оцениваем. Analyzer может быть полностью корректен как parser, но иметь неполный набор эвристик для некоторых вмешательств. Он может не давать false positives на benign corpus, но это не доказывает recall на неизвестных obfuscation cases. Он может валидно формировать JSON, но это отдельное качество report contract, а не detection accuracy.

Поэтому правильный ответ начинается с уточнения: “точность чего именно?”. Для EH/Unwind Analyzer есть минимум четыре разных уровня оценки: file-level, record-level, finding-level и event-level. Если смешать их в одно число, получится красивая, но научно слабая цифра.

Уровень	Вопрос	Почему отдельный
File-level	Должен ли PE получить хотя бы один тревожный сигнал?	Просто для презентации, но грубо: файл с одной проблемой и файл с сотней проблем считаются одинаково.
Record-level	Правильно ли отмечена конкретная RUNTIME_FUNCTION или UNWIND_INFO?	Подходит для проверки parser-а и .pdata/.xdata logic.
Finding-level	Должен ли конкретный code из 27 появиться в отчете конкретного файла?	Лучший уровень для оценки 27 признаков: precision/recall можно считать per finding.
Event-level	Должно ли dynamic событие быть зафиксировано при выполнении?	Нужно для VEH/VCH/UEF/RtlAddFunctionTable/handler hit.

2. Что такое качество проекта

Качество продукта шире, чем detection accuracy. Для инструмента reverse engineering качество означает: он не падает на поврежденных файлах, правильно читает формат, объясняет вывод, сохраняет машинно-читаемый отчет, воспроизводимо работает через CLI, имеет тестовый corpus, умеет отделять жесткие structural violations от контекстных suspicion и честно показывает границы своих выводов.

Ось качества	Что проверяется	Как доказывать
Functional correctness	Функции проекта делают то, что заявлено: читают PE, .pdata, .xdata, формируют findings.	Unit tests, regression samples, expected findings.
Parser safety	Нет чтения за пределами, переполнений, crash на malformed input.	Bounds checks, mutation fuzz, partial reports.
Detection quality	Признаки срабатывают на правильных нарушениях и не срабатывают на нормальных случаях.	TP/FP/TN/FN на размеченном corpus.
Report quality	Отчет показывает evidence, matrix 27 features, key diagnostics, JSON schema.	unit_report_json/html, semantic validator, visual QA.
Dynamic validity	Runtime-события фиксируются в Windows dynamic mode.	Instrumented dynamic samples, event-level validation.
Usability	CLI понятен, exit codes предсказуемы, отчеты можно открыть и передать backend/frontend.	cli_contract, documentation, examples.
Maintainability	Код разделен на parser, heuristics, report, dynamic, tests.	Архитектура, CTest, release hygiene.
Release readiness	Поставка чистая, версии schema согласованы, лишние артефакты отсутствуют.	release_metadata, release_hygiene.

3. Ground truth

Ground truth - это эталонная разметка. Без нее нельзя честно считать TP, FP, TN, FN. В нашем проекте ground truth может быть создан разными способами: специально сконструированный PE, где мы точно знаем внесенное нарушение; ручная экспертная проверка; comparison с независимым parser-ом; сведения о compiler/linker provenance; instrumented dynamic target, который специально вызывает нужный handler или API.

Источник ground truth	Плюс	Минус
Synthetic sample	Точно известно, что именно сломано.	Не покрывает все разнообразие реальных toolchains.

Источник ground truth	Плюс	Минус
Expert manual review	Высокое доверие к сложным случаям.	Дорого, медленно, возможны расхождения экспертов.
Differential parsers	Можно сравнить с независимыми инструментами.	Инструменты тоже могут ошибаться.
Compiler provenance	Известно, чем и как собран benign sample.	Не всегда доступно для реальных PE.
Instrumented dynamic target	Можно точно ожидать VEH/UEF/RtlAddFunctionTable event.	Покрывает только контролируемый runtime path.

Фраза для комиссии
Нельзя говорить о false positive или false negative без ground truth. Если эталонной разметки нет, корректно говорить: “finding требует экспертной проверки”, а не “это точно ошибка инструмента” или “это точно атака”.

4. Confusion matrix

Confusion matrix - таблица четырех исходов сравнения результата analyzer-a с ground truth. Она применима не только в машинном обучении. Любой детектор, который выдает положительный/отрицательный сигнал, можно оценить через TP, FP, TN, FN.

Ground truth / результат	Analyzer дал сигнал	Analyzer не дал сигнал
Нарушение есть	TP: true positive, правильное обнаружение.	FN: false negative, нарушение пропущено.
Нарушения нет	FP: false positive, ложное срабатывание.	TN: true negative, правильное отсутствие сигнала.

В проекте “сигнал” лучше определять не как “файл вредоносный”, а как более узкое событие: конкретный finding code появился в отчете. Например, для finding-level оценки code rf_overlaps является сигналом, а ground truth говорит, есть ли реальное пересечение RUNTIME_FUNCTION ranges.

5. Метрики точности

Метрика	Формула	Смысл
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Доля правильных решений среди всех. Уязвима к дисбалансу классов.
Precision	TP / (TP + FP)	Когда инструмент сработал, насколько часто срабатывание подтверждается.
Recall / sensitivity	TP / (TP + FN)	Когда нарушение реально есть, насколько часто инструмент его находит.
Specificity	TN / (TN + FP)	Когда нарушения нет, насколько часто инструмент не создает ложный сигнал.
False Positive Rate	FP / (FP + TN)	Доля ложных тревог среди нормальных объектов.
False Negative Rate	FN / (FN + TP)	Доля пропусков среди объектов с реальным нарушением.
F1-score	2 * precision * recall / (precision + recall)	Баланс precision и recall; не учитывает TN.
NPV	TN / (TN + FN)	Если инструмент молчит, насколько часто это действительно нормальный случай.

Почему accuracy опасна: если в корпусе 990 benign и 10 anomalous samples, инструмент, который всегда отвечает “аномалий нет”, получит 99% accuracy, но recall по нарушениям будет 0%. Для triage-инструмента это неприемлемо. Поэтому на защите нужно говорить не только accuracy, а precision, recall, specificity и false-positive rate.

6. Score, confidence и accuracy - разные вещи

Понятие	Что это	Чем не является
Score	Внутренняя численная оценка подозрительности по найденным признакам.	Не процент вредоносности и не accuracy.
Confidence	Оценка силы evidence и надежности вывода в рамках эвристической модели.	Не ML probability.

Понятие	Что это	Чем не является
Verdict	Текстовая категория итогового вывода: low/moderate/high suspicion и т.п.	Не судебный приговор о вредоносности.
Accuracy	Внешняя экспериментальная метрика на размеченном corpus.	Не поле, которое analyzer может честно вывести сам без ground truth.
Finding	Технический сигнал о нарушении/аномалии.	Не утверждение “файл вредоносный”.

Если комиссия спросит, почему в отчете есть confidence, но ты говоришь, что accuracy не заявляется, ответ такой: confidence - это внутренняя эвристическая уверенность в evidence конкретного анализа. Accuracy - это внешнее статистическое свойство инструмента на benchmark. Это разные уровни.

7. Текущие evidence качества в проекте

В проекте есть несколько уже подготовленных доказательств качества. Они не равны глобальной accuracy, но являются сильными инженерными validation signals.

Evidence	Факт	Что доказывает	Что не доказывает
CTest	16/16 tests passed в последнем локальном прогоне.	Кодовые сценарии parser/heuristics/reports/CLI/fuzz/release проходят.	Не доказывает 100% detection accuracy на всех PE.
Extended corpus	78 PE: 60 static, 9 dynamic, 9 mixed.	Есть controlled samples для демонстрации и регрессии.	Не является большим реальным benchmark.
Reports	78 HTML, 78 JSON, 78 CSV, 78 Ghidra-CSV.	Выходные форматы формируются воспроизводимо.	Не доказывает экспертную истинность каждого вывода.
Matrix 27 features	HTML показывает все 27 признаков: active/inactive.	Покрытие отчетности по диагностической модели.	Не значит, что 27 признаков покрывают все возможные атаки.
Mutation fuzz	120 deterministic mutations внутри CTest.	Устойчивость к поврежденным входам.	Не является detection benchmark.
Release validation	В release-документах: 14/14 static labeled valid; 106/106 real benign valid; non-zero scores 0; 174795 runtime functions parsed.	Показывает отсутствие наблюдаемых FP-score на данном benign corpus и масштаб parsing validation.	Не переносится автоматически на все benign PE мира.

8. Как объяснять 16 тестов, 27 признаков и 78 бинарников

Это частый вопрос. 16 - количество CTest-сценариев. 27 - количество диагностических признаков. 78 - количество тестовых PE-входов в расширенном корпусе. Эти числа не обязаны совпадать, потому что один тестовый сценарий может проверять несколько признаков, а один PE может активировать несколько findings.

Число	Что означает	Роль в качестве
16	Автоматизированные CTest-сценарии.	Quality gate для кода и контрактов.
27	Эвристические признаки модели анализа.	Функциональная ширина диагностики.
78	Тестовые PE-бинарники.	Входной corpus для демонстрации и регрессии.
120	Mutation fuzz cases.	Robustness к поврежденным входам.
106	Benign real corpus из release-документов.	Проверка ложных тревог на конкретном benign наборе.

9. Как считать качество static-части

Класс проверки	Что проверяет	Метрика качества
PE layout correctness	raw/virtual OOB, overlaps, alignment, Exception Directory validity.	Precision/recall по structural findings; parser failure rate.
.pdata correctness	invalid ranges, function OOB, unsorted, overlaps, duplicate begin.	Record-level TP/FN по RUNTIME_FUNCTION entries.
.xdata correctness	UNWIND_INFO Version/Flags/CountOfCodes/UWOP slots/padding/tail.	Record-level parse accuracy и finding-level recall.
Placement/context	writable/executable metadata, handler nonexec/writable, unusual section.	Precision важна, потому что редкие benign layouts могут давать FP.

Класс проверки	Что проверяет	Метрика качества
Reuse/dominance	unwind reuse, handler dominance, combo rules.	Нужна benign calibration, чтобы central language handlers не давали лишние тревоги.
Report evidence	RVA, section, file offset, expected/available, failed_field.	Evidence completeness, schema validity, semantic consistency.

10. Как считать качество dynamic-части

Класс dynamic	Что проверяет	Метрика качества
Session reliability	ran, ok, timed_out, target_exited, instrumentation_complete.	Success rate dynamic sessions; timeout/failure rate.
Exception observation	exceptions_seen, first_chance_seen, second_chance_seen.	Event visibility на instrumented targets.
Handler observation	unwind_handler_hits, veh_hits, vch_hits, uef_hits.	Event-level recall: ожидаемый handler hit был зафиксирован или нет.
Runtime function tables	RtlAddFunctionTable, RtlInstallFunctionTableCallback.	Recall по instrumented dynamic function table samples.
Event evidence quality	events array: type/module/thread/address/args.	Полнота динамического отчета.
Trace quality	RIP trace после handler hit.	Наличие route evidence, но не полная реконструкция control flow.

11. False positives: что это и как снижать

False positive - ситуация, когда finding появился, но ground truth говорит, что нарушения нет. Для проекта это особенно важно, потому что инструмент предназначен для реверсеров: лишние тревоги создают ручную нагрузку.

Причина FP	Пример	Как снижать
Редкий compiler/toolchain layout	Metadata лежит не в привычной секции, но это допустимо.	Контекстные правила, severity, benign corpus calibration.
Protector/runtime без злого смысла	Central handler обслуживает много функций.	Handler dominance не считать автоматически атакой; показывать evidence.
JIT/runtime metadata	RtlAddFunctionTable используется легитимно.	Dynamic validation и manual interpretation.
Слишком жесткий threshold	Слабый сигнал стал сильным verdict.	Threshold tuning, weakening rules.
Неполный контекст отчета	Finding есть, но не показаны benign alternatives.	Добавлять explanation, next step, confidence.

В текущем проекте FP снижаются тем, что отчет не говорит “malware”, а показывает technical finding, severity, evidence, context и manual validation steps. Это корректная triage-позиция.

12. False negatives: что это и как снижать

False negative - ситуация, когда нарушение реально есть, но analyzer его не обнаружил. Для проекта это возможно, если вмешательство не нарушает текущие инварианты или находится вне зоны видимости static/dynamic слоя.

Причина FN	Пример	Как снижать
Согласованное изменение metadata	RUNTIME_FUNCTION и UNWIND_INFO изменены так, что формально выглядят корректно.	Добавить disassembly/prolog matching и semantic validation.
Нет проверки .text	UWOP заявляет одно, prolog делает другое.	Добавить Capstone/Zydis-like disassembler layer.
Runtime path не достигнут	Handler существует, но sample run не вызвал нужное исключение.	Targeted dynamic tests и input сценарии.
Ненаблюдаемый registration mechanism	Handler/function table регистрируется способом вне текущих breakpoints.	Расширить dynamic instrumentation.
Новый/редкий формат	Неподдержанная toolchain-specific metadata.	Расширить parser и corpus.

13. Performance, portability, security как качество

Комиссия может спросить не только про точность. Качество продукта также включает скорость, переносимость и безопасность использования.

Направление	Что измерять	Как отвечать
Performance	Время анализа файла, файлов/минуту, MB/s, память.	Сейчас основной упор на корректность и evidence; performance можно измерить batch-прогоном corpus.
Portability	Static на Windows/Linux/macOS, dynamic только Windows.	Static работает с PE как файлом на разных ОС; dynamic зависит от Windows Debug API.
Security	Не исполнять target в static mode; dynamic запускать в lab/sandbox.	Static безопаснее; dynamic требует контролируемой среды.
Reliability	Exit codes, partial reports, atomic writes.	cli_contract проверяет предсказуемое поведение.
Integration	JSON schema stability, CSV/Ghidra CSV.	JSON - контракт для backend/DB/frontend.

14. Как построить строгий benchmark

1. Определить задачу

Например: finding-level detection для каждого из 27 codes, а не “вредоносный/не вредоносный”.

2. Определить positive/negative classes

Для rf_overlaps positive - реальные пересечения ranges; negative - корректные непересекающиеся ranges.

3. Собрать corpus

Synthetic positives, benign negatives, real-world PE, compiler-known samples, protector/JIT samples.

4. Разметить ground truth

Записать, что именно сломано, где RVA, кто проверил, чем подтверждено.

5. Зафиксировать команды запуска

Одна версия analyzer, одинаковые флаги, сохранение JSON outputs.

6. Посчитать confusion matrix

TP/FP/TN/FN отдельно для каждого finding code и каждого уровня оценки.

7. Посчитать metrics

Precision, recall, specificity, FPR, FNR, F1; отдельно для static и dynamic.

8. Разобрать ошибки

Каждый FP и FN превращается в issue: правило, threshold, parser, corpus, документация.

9. Повторить после правок

Regression testing не должен ломать уже закрытые классы.

15. Пример расчета метрик

Допустим, мы оцениваем finding rf_overlaps на finding-level. Есть 40 PE с настоящим overlap и 100 PE без overlap. Analyzer нашел overlap в 38 positive PE и ошибочно нашел overlap в 2 negative PE. Тогда: TP=38, FN=2, FP=2, TN=98.

```
precision = 38 / (38 + 2) = 95%
recall = 38 / (38 + 2) = 95%
specificity = 98 / (98 + 2) = 98%
accuracy = (38 + 98) / 140 = 97.14%
```

Но если для другого finding, например handler_dominance, precision ниже из-за легитимных central handlers, общий средний процент может скрыть эту проблему. Поэтому метрики надо считать per finding class.

16. Как отвечать на жесткие вопросы

Как вы оцениваете качество проекта?

Многоуровнево: CTest, regression samples, extended PE corpus, schema/semantic validation, mutation fuzz, benign corpus checks, dynamic validation и качество evidence в отчетах.

Какая точность?

Глобальную accuracy не заявляю без большого размеченного benchmark. Для текущей версии есть инженерная валидация; строгая точность должна считаться через TP/FP/TN/FN per finding class.

Почему нет одного процента?

Потому что parsing accuracy, finding recall, benign specificity, dynamic event recall и report validity - разные свойства. Один процент смешал бы разные задачи.

Можно ли сказать, что false positives отсутствуют?

Только на конкретном проверенном benign corpus при данном threshold. Нельзя утверждать отсутствие FP во всей мировой совокупности PE.

Можно ли сказать, что false negatives отсутствуют?

Нет. FN возможны, если нарушение не попадает в текущие инварианты или runtime path не достигнут. Это честное направление развития.

Что сильнее всего доказывает качество?

Traceability: требование -> test sample -> expected finding -> actual JSON/HTML -> automated validation -> readable evidence.

Почему это не ML accuracy?

Потому что нет обучаемой модели. Признаки rule-based. ML-метрики используются только для внешней оценки детектора на размеченном corpus.

Что бы вы улучшили для измерения качества?

Собрал бы большой per-finding benchmark, добавил record-level разметку, disassembly/prolog matching, Windows dynamic CI и расчет precision/recall по каждому классу.

17. Готовый ответ на 2 минуты

“Качество проекта я оцениваю не одним процентом, потому что это не нейросетевой классификатор вредоносности, а инженерный анализатор PE x64 EH/Unwind metadata. У него несколько разных уровней качества: корректность парсера, корректность findings, отсутствие ложных тревог на benign corpus, устойчивость к поврежденным PE, валидность отчетов, dynamic validation и релизная готовность. Последний локальный gate прошел 16/16 CTest; расширенный корпус содержит 78 PE и по ним сгенерированы HTML/JSON/CSV/Ghidra-CSV; в release-документах также зафиксированы проверки labeled и benign corpus.”

“Если говорить строго о точности, ее надо считать на размеченном benchmark. Для каждого finding code из 27 нужно определить ground truth и посчитать TP, FP, TN, FN. Затем отдельно считать precision, recall, specificity, false-positive rate и false-negative rate. Я сознательно не называю необоснованную 100% accuracy, потому что это было бы некорректно без большого репрезентативного corpus. Вместо этого

проект показывает воспроизводимую инженерную валидацию и готовый план строгой оценки качества.”

18. Короткая шпаргалка

Если спрашивают	Ответ в одну фразу
Качество?	Многоуровневая инженерная валидация: parser, heuristics, reports, CLI, fuzz, corpus, dynamic.
Точность?	Строго считается через TP/FP/TN/FN на размеченном benchmark; глобальную accuracy без benchmark не заявляю.
False positives?	Возможны; снижаются context rules, severity, confidence, benign validation и manual review.
False negatives?	Возможны; снижаются расширением правил, disassembly/prolog matching и targeted dynamic.
Почему не ML?	Форматные инварианты лучше проверяются rule-based и объяснимо; ML можно добавить только как triage layer.
Что доказано?	Текущий quality gate и corpus validation; не абсолютная мировая точность.

19. Итог

Сильная позиция на защите: проект имеет проверяемое инженерное качество, но его точность надо описывать строго. Не обещать невозможное, а показывать evidence. Не говорить “100% accuracy”, а говорить “16/16 CTest passed, 78 PE corpus processed, reports validated, benign corpus checks, mutation fuzz, and a defined methodology for TP/FP/TN/FN measurement”. Это звучит профессионально и защищает от попытки комиссии поймать на некорректной статистике.

ПРИЛОЖЕНИЕ: FINDINGS, SCORE И ОБОСНОВАНИЕ ВЕСОВ

Что важно сказать комиссии
Число score в отчете - это не процент вредоносности, не ML probability и не статистическая accuracy. Это инженерная шкала 0..100, которая суммирует силу обнаруженных технических признаков. Чем сильнее нарушение формата и чем больше независимых семейств evidence, тем выше score.

20. Как устроен score

В коде проекта score хранится как целое число от 0 до 100. Каждое finding добавляет фиксированный или условный вес. После сложения значение ограничивается диапазоном 0..100. Затем suspicion вычисляется как score / 100. Это удобно для отчета, но это не означает вероятность атаки. Например, score 25 означает “накоплено 25 баллов статической подозрительности”, а не “25% файл вредоносный”.

Поле отчета	Как считается	Как объяснять
score	Сумма весов сработавших признаков, clamp 0..100.	Инженерная оценка силы статических сигналов.
suspicion	score / 100.	Нормированное представление score, а не вероятность.
verdict	>=70 high, >=40 moderate, >=15 low, иначе no strong anomalies.	Категория для triage, а не окончательный диагноз.
evidence_level	strong/medium/weak по числу high/medium findings и score.	Сила evidence, а не accuracy.
confidence	База + бонусы за семьи/сильные признаки - penalties.	Насколько надежно выглядит вывод в рамках правил.

21. Почему веса именно такие

Веса сгруппированы по смыслу evidence. Самые большие веса получают hard structural/integrity violations: они ломают базовую карту PE/.pdata/.xdata и могут сделать unwind ненадежным. Средние веса получают placement anomalies: они подозрительны, но могут иметь benign alternatives. Небольшие веса получают context/combo-признаки или одиночные слабые сигналы: они усиливают картину, но редко должны быть единственным основанием для сильного вывода.

Группа	Типичный вес	Почему
Жесткие нарушения PE/.pdata/.xdata	20-25	Ломают маппинг, диапазоны функций или чтение UNWIND_INFO; это сильное техническое evidence.
Синтаксические anomalies UNWIND_INFO	10-25	Зависят от распространенности: единичный сбой слабее массового сбоя.
Placement anomalies	8-20	Подозрительны, но возможны редкие benign layout/runtime cases.
Reuse/dominance	10-20	Может быть нормальным runtime/language pattern, поэтому требует контекста.
Combo-признаки	8-10	Не заменяют базовые findings, а усиливают связку нескольких сигналов.

22. Пороги процентов и ratios

В scoring есть не только фиксированные веса, но и пороги распространенности. Это сделано, чтобы отличать одиночную локальную проблему от системного нарушения по множеству записей.

Порог	Где используется	Смысл
>1% entries	invalid_ranges, rf_oob, unwind_rva_oob, unwind_parse_fail.	Если нарушение затрагивает больше 1% .pdata, оно считается не случайной единичной проблемой, а более широкой деградацией карты.
>5% entries	unwind_anomalies, unusual/writable placement high weight.	Массовость anomalies/placement делает сигнал сильнее.
>10% reuse и n > 200	unwind_reuse_moderate.	Умеренный reuse значим только на достаточно большой таблице и при наличии другого сигнала.
>30% reuse и n > 200	unwind_reuse_strong.	Один UNWIND_INFO обслуживает слишком большую долю функций; это сильный pattern.

Порог	Где используется	Смысл
handler_count > 50 и reuse > 80%	handler_dominance.	Один handler доминирует над большим числом функций; без контекста это suspicious dispatcher-like pattern.

23. Полная таблица weights по 27 findings

Finding code	Вес	Severity	Почему такая оценка
pe_section_raw_oob	25	high	Raw-диапазон секции выходит за файл. Это ломает доверие к чтению байтов.
pe_section_virtual_oob	20	high	Virtual-диапазон секции выходит за SizeOfImage. Это ломает карту RVA image.
pe_section_raw_overlap	15	medium	Raw-диапазоны секций пересекаются. Сигнал серьезный, но raw overlap может быть менее критичен, чем virtual ambiguity.
pe_section_virtual_overlap	20	high	RVA-диапазоны пересекаются. Один RVA может соответствовать нескольким секциям, поэтому это сильнее raw overlap.
pe_section_misaligned	8	medium	Alignment/header invariant нарушен. Вес ниже, потому что часть нестандартных файлов может иметь странный, но не атакующий layout.
pe_exception_directory_oob	25	high	Exception Directory не читается корректно. Для x64 unwind это вход в .pdata, поэтому вес максимальный.
no_runtime_functions	10	medium	В x64 image с executable code отсутствие .pdata может быть допустимо для leaf-only/simple image, поэтому вес умеренный.
invalid_ranges	10/20	medium/high	BeginAddress >= EndAddress. Если >1% записей, вес 20; единичное нарушение весит 10.
rf_oob	10/25	medium/high	Function range выходит за SizeOfImage. Массовость >1% повышает вес до 25.
unwind_rva_oob	10/25	medium/high	UnwindInfoAddress не ведет к допустимым байтам. Массовость >1% повышает вес до 25.
rf_unsorted	15	medium	Таблица не отсортирована. Это мешает однозначному поиску, но не всегда означает corruption metadata.
rf_overlaps	25	high	Диапазоны функций пересекаются. Один ControlPc может получить неоднозначную unwind-запись.
duplicate_begin	10	medium	Повтор BeginAddress подозрителен, но может требовать контекста, поэтому вес умеренный.
unwind_parse_fail	10/25	medium/high	UNWIND_INFO не разбирается. Если >1% записей, вес 25, потому что metadata системно ненадежна.
unwind_anomalies	10/20	medium/high	UNWIND_INFO читается, но поля/операции странные. Если >5% записей, вес повышается.
unwind_reuse_strong	20	high	>30% функций при n>200 используют один UnwindInfoRVA. Это сильный pattern централизованной metadata.
unwind_reuse_moderate	10	medium	>10% reuse при n>200, но только если есть другой static signal. Сам по себе умеренный reuse не пугает.
handler_dominance	15	medium	Один handler обслуживает >80% handlers при handler_count>50. Вес средний из-за возможных language/runtime handlers.
unwind_in_exec_section	20	high	Metadata лежит в executable section. Это смешивает код и metadata, что подозрительно.
unwind_unusual_section	8/15	medium/high	UNWIND_INFO в нетипичной секции. Если >5% записей, вес 15; иначе 8.
unwind_writable_section	10/18	medium/high	Metadata находится в writable section. Массовость >5% повышает вес до 18.
handler_nonexec_section	15	high	Handler RVA ведет в non-executable/unknown область. Handler должен быть кодом.
handler_writable_section	10	medium	Handler target writable. Подозрительно, но может требовать dynamic/manual проверки.
placement_exec_nonexec_handler_combo	10	high	Executable metadata + non-executable handler вместе сложнее объяснить benign layout.
placement_integrity_combo	8	medium	Suspicious placement подтверждается integrity-проблемами. Это усилитель, а не базовая причина.
reuse_dominance_combo	8	medium	Unwind reuse + handler dominance похожи на dispatcher-style pattern.
writable_metadata_handler_combo	10	high	Writable metadata + writable handler target вместе являются сильной необычной связкой.

24. Confidence: почему может снижаться

Score показывает силу сигналов, но confidence показывает, насколько надежен сам вывод. Confidence начинается с базы 0.15. При нулевом score confidence ставится 0.20 как “нет признаков, но это не доказательство полной чистоты”. Затем confidence растет за разные evidence families, high/medium findings и hard table/unwind problems. Но confidence снижается, если сигнал одиночный, sample маленький, evidence только из placement или только из одного семейства.

Элемент confidence	Значение	Смысл
Base	+0.15	Минимальная стартовая уверенность при наличии анализа.
Zero score base	0.20	Если score=0, это аккуратная уверенность в отсутствии сильных сигналов, не абсолютная чистота.
Per evidence family	+0.11	Разные семейства evidence повышают доверие: table, unwind, placement, reuse, dominance.
Per strong finding	+0.08	High/critical findings усиливают вывод.
Per medium finding	+0.04	Medium findings добавляют умеренную уверенность.
Hard table bonus	+0.08	Invalid ranges/OOB/overlaps - жесткие проблемы карты .pdata.
Hard unwind bonus	+0.08	Parse failures/anomalies - жесткие проблемы .xdata.
Single signal penalty	-0.12	Один несильный сигнал не должен давать чрезмерную уверенность.
Small sample penalty	-0.06	Малая .pdata может завышать ratios.
Placement-only penalty	-0.10	Placement-only может быть benign uncommon layout.
Single-family penalty	-0.08	Один тип evidence слабее cross-family correlation.
Bounds	0.05..0.90	Confidence никогда не становится 0 или 1.

25. Примеры объяснения score

Пример 1. Если sample 77 имеет unwind_parse_fail из-за CountOfCodes/UWOP operand slots и это затрагивает 2 из 3 записей, доля больше 1%, поэтому finding получает high branch: +25. Итоговый score может быть 25, verdict low_suspicion, потому что это один основной класс проблемы: серьезная ошибка metadata есть, но нет множества независимых семейств evidence.

Пример 2. Если один invalid_range найден в таблице из 300 записей, это 0.33%, то применяется low branch +10. Если invalid ranges затрагивают больше 1%, применяется high branch +20. Логика такая: единичная поврежденная запись слабее массовой деградации таблицы.

Пример 3. Writable metadata (+10 или +18) и writable handler (+10) вместе дополнительно дают writable_metadata_handler_combo (+10). Комбинация важна, потому что одна writable metadata может быть редкостью toolchain-a, но writable metadata плюс writable handler target выглядит значительно более подозрительно.

Короткий ответ комиссии
Веса назначены не случайно. Они отражают инженерную силу evidence: hard structural/unwind violations получают высокий вес, контекстные placement/reuse признаки - средний, combo-признаки - дополнительный усилитель. Пороги 1%, 5%, 10%, 30% и 80% нужны, чтобы отличать одиночные аномалии от системного паттерна.

FALSE POSITIVE, FALSE NEGATIVE И ТОЧНОСТЬ

Как объяснять срабатывания EH/Unwind Analyzer на защите

Документ объясняет TP, FP, TN, FN, precision, recall, accuracy, ground truth и то, почему проект не является ML-моделью, хотя эти термины применимы к его оценке качества.

Главный ответ комиссии
Внутри проекта нет машинного обучения и нет обучаемой модели. Анализатор работает по правилам и эвристикам. Поэтому TP/FP/TN/FN используются не как часть работы алгоритма, а как язык оценки качества на размеченном корпусе: сравниваются ожидаемые нарушения и фактические findings отчета.

1. Есть ли это в проекте

В текущем коде EH/Unwind Analyzer нет ML-классификатора, обучения, feature vectors в ML-смысле, training set, loss function и автоматического расчета confusion matrix. Проект использует детерминированные проверки и эвристические правила: если нарушен структурный или контекстный инвариант, создается finding.

Тем не менее термины true positive, false positive, true negative и false negative применимы к проекту при внешней оценке качества. Для этого нужен размеченный corpus: для каждого файла, записи или признака заранее известно, должно ли нарушение считаться присутствующим. Потом результат анализатора сравнивается с этой эталонной разметкой.

Что	Внутри проекта сейчас	Как использовать на защите
ML-модель	Нет	Говорить честно: это rule-based/static+dynamic analyzer, а не нейросетевая модель.
27 признаков	Да	Это набор диагностических правил, а не обученные веса модели.
Score/confidence	Да	Эвристическая оценка силы evidence, не ML probability.
TP/FP/TN/FN	Не считаются автоматически как runtime-метрика	Их можно считать при экспериментальной оценке на размеченном корпусе.
Accuracy/precision/recall	Не являются встроенным итогом продукта	Их можно получить только после строгой разметки positive/negative samples.

2. Базовые термины

Термин	Перевод	Смысл для проекта
Ground truth	Эталонная разметка	Заранее установленная истина: какое нарушение есть в файле, где оно находится и почему оно считается нарушением.
Positive	Положительный класс	Случай, где нарушение или интересующий finding действительно должен быть обнаружен.
Negative	Отрицательный класс	Случай, где нарушения данного типа нет и finding появляться не должен.
Signal / finding	Срабатывание / диагностический вывод	То, что analyzer записал в отчет: например unwind_parse_fail или rf_overlaps.
Threshold	Порог решения	Граница, после которой score или набор признаков считается достаточным для тревожного вывода.

Без ground truth нельзя честно говорить “false positive” или “false negative”. Если эксперт не знает, было ли нарушение на самом деле, можно говорить только “finding требует проверки” или “подозрительный сигнал”, но не вычислять точность.

3. Confusion matrix

Confusion matrix - это таблица четырех вариантов результата для бинарного решения. В нашем проекте бинарным решением может быть не “вредоносный файл или нет”, а более узкий вопрос: например, должен ли сработать конкретный finding code для конкретного PE.

Ground truth / результат	Analyzer дал сигнал	Analyzer не дал сигнал
Нарушение есть	TP: true positive. Правильное обнаружение.	FN: false negative. Нарушение пропущено.
Нарушения нет	FP: false positive. Ложное срабатывание.	TN: true negative. Правильное отсутствие сигнала.

4. Что такое TP, FP, TN, FN на примерах проекта

Случай	Пример для EH/Unwind Analyzer	Как объяснять
TP	В тестовом PE действительно пересекаются RUNTIME_FUNCTION ranges, и отчет содержит rf_overlaps.	Инструмент правильно обнаружил заранее внесенное нарушение.
FP	Корректный PE нестандартного toolchain получает finding handler_dominance, хотя это нормальный central language handler.	Это ложное срабатывание; его уменьшают context rules, weakening rules и ручная проверка.
TN	Benign PE имеет обычные .pdata/.xdata, и отчет не содержит тревожных findings.	Инструмент правильно не создал лишнюю тревогу.
FN	В PE есть согласованно измененная metadata, но она не нарушает текущие 27 инвариантов, и finding не появился.	Это пропуск относительно более широкой цели; лечится расширением правил, disassembly/prolog matching и dynamic correlation.

5. Метрики качества

Метрика	Формула	Что означает
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Доля всех правильных решений. Опасна при сильном дисбалансе classes.
Precision	TP / (TP + FP)	Из всех срабатываний какая доля действительно подтверждается ground truth.
Recall / sensitivity	TP / (TP + FN)	Из всех реальных нарушений какую долю инструмент нашел.
Specificity	TN / (TN + FP)	Из всех нормальных случаев какую долю инструмент не пометил ошибочно.
FPR	FP / (FP + TN)	False positive rate: доля ложных тревог среди negative cases.
FNR	FN / (FN + TP)	False negative rate: доля пропусков среди positive cases.
F1	2 * precision * recall / (precision + recall)	Баланс precision и recall, но скрывает TN и не заменяет полный анализ.

Для защиты важно: overall accuracy может выглядеть красиво, но быть бесполезной. Если corpus почти весь benign, инструмент, который всегда говорит “аномалий нет”, получит высокую accuracy, но recall по нарушениям будет нулевой. Поэтому для такого анализатора лучше говорить о per-class precision/recall по конкретным finding families.

6. Почему это не машинное обучение

В машинном обучении модель обычно обучается на dataset: получает признаки, оптимизирует параметры, затем на test set оценивается через TP/FP/TN/FN, accuracy, precision, recall. В нашем проекте нет обучения параметров. Признаки выбраны инженерно: они основаны на структуре PE x64 EH/Unwind metadata, требованиях формата, ожидаемых свойствах RUNTIME_FUNCTION/UNWIND_INFO и подозрительных сочетаниях placement/reuse/handler behavior.

Поэтому фраза “точность модели” опасна. Если комиссия спросит, нужно уточнить: “Если под моделью понимать ML-модель, ее нет. Если под моделью понимать эвристическую модель анализа, ее качество оценивается через тестовый и размеченный corpus, но глобальная statistical accuracy без репрезентативного ground truth corpus не заявляется”.

Готовый ответ
Проект не обучается и не классифицирует вредоносность как нейросеть. Он выявляет структурные и контекстные аномалии EH/Unwind metadata. Термины TP/FP/TN/FN применяются на этапе проверки качества: мы сравниваем ожидаемые findings на размеченных PE с фактическими findings отчета.

7. Какие validation signals есть сейчас

Проверка	Что показывает	Чего не доказывает
16/16 CTest	Кодовые сценарии парсера, эвристик, отчетов, CLI, fuzz и release checks проходят.	Не является статистической accuracy на мировом PE-corpus.
78 тестовых PE	Подготовленный корпус демонстрирует normal/static/dynamic/mixed cases и expected findings.	Не заменяет размеченный real-world benchmark.
Real benign corpus 106/106 valid, non-zero scores 0	На проверенном benign-наборе не наблюдались false-positive scores.	Не гарантирует отсутствие FP на всех benign PE мира.
Static labeled corpus 14/14 valid	На размеченных synthetic/labeled случаях отчеты валидны и ожидаемые сценарии проходят.	Размер corpus мал для общей статистики точности.
Mutation fuzz 120 cases	Парсер устойчив к поврежденным входам: controlled failure или valid report.	Это robustness, а не detection accuracy.

8. Как правильно строить эксперимент для TP/FP/TN/FN

- Определить unit of evaluation: file-level, record-level, finding-level или event-level.
- Задать positive class: например, “есть rf_overlaps” или “есть malformed CountOfCodes”.
- Задать negative class: корректные объекты без данного нарушения.
- Создать ground truth: synthetic samples, ручная экспертная разметка, differential parsers, compiler provenance.
- Запустить analyzer одинаковой командой и сохранить JSON/HTML evidence.
- Для каждого объекта сравнить expected finding с actual finding.
- Посчитать TP, FP, TN, FN и затем precision, recall, specificity, FPR, FNR.
- Разбирать FP и FN отдельно: это источник улучшения правил, thresholds, context mitigations и документации.

9. Как отвечать на жесткие вопросы

Какая точность проекта?

Общая статистическая accuracy не заявляется, потому что для нее нужен большой репрезентативный размеченный corpus. Есть инженерная валидация: CTest, synthetic/labeled corpus, benign corpus, dynamic corpus и mutation fuzz.

Есть ли false positives?

Возможны, как у любого эвристического анализатора. Проект снижает их через evidence, severity, confidence, context rules, weakening rules и требование ручной проверки.

Есть ли false negatives?

Возможны. Например, если вмешательство не нарушает проверяемые инварианты или код и metadata изменены согласованно. Для снижения FN нужны disassembly/prolog matching и расширение dynamic correlation.

Почему не используете нейросеть?

Потому что задача основана на формальных структурах PE/EH/Unwind. Для таких инвариантов rule-based подход объяснимее, воспроизводимее и лучше защищается evidence. ML можно использовать позже для triage, но не вместо строгого parser-а.

Можно ли считать 106/106 benign как 100% specificity?

Только на этом конкретном corpus и при выбранном threshold. Нельзя переносить это на все возможные benign PE без дополнительного benchmark.

Что значит confidence в отчете?

Это эвристическая уверенность/сила evidence в рамках правил проекта, а не ML probability и не вероятность вредоносности.

10. Итоговая формулировка для защиты

“В проекте нет машинного обучения, поэтому TP/FP/TN/FN не используются как внутренняя логика модели. Анализатор rule-based: он проверяет структурные и контекстные инварианты PE x64 EH/Unwind metadata. Но при оценке качества эти термины применимы: на размеченном corpus мы сравниваем expected findings с actual findings. True positive - нарушение было и найдено; false positive - нарушения не было, но finding появился; true negative - нарушения не было и finding не появился; false negative - нарушение было, но analyzer его пропустил. Общую accuracy я не заявляю без большого ground truth corpus; вместо этого показываю многоуровневую инженерную валидацию и объяснимые отчеты с evidence.”